

Федеральное государственное автономное образовательное учреждение
высшего образования «Самарский национальный исследовательский
университет имени академика С.П. Королева»

Отчёт

по лабораторной работе №7

Выполнил:

студент группы

6301-030301D

Панин Ю. В.

Проверил:

Борисов Д. С.

Самара, 2025

Содержание

Содержание	2
Задание 1	2
Задание 2	5
Задание 3	5

Задание 1

Цель: сделать так, чтобы все объекты типа `TabulatedFunction` можно было использовать в качестве объекта-агрегата в цикле `for-each`, извлекаемые объекты при этом должны иметь тип `FunctionPoint`.

В интерфейсе `TabulatedFunction` добавлен необходимый родительский тип, при этом использован параметризованный тип (generic type).

В классах, реализующих интерфейс `TabulatedFunction`, добавлен требуемый метод, возвращающий объект итератора.

Классы итераторов сделаны анонимными. В соответствии с паттерном «Итератор» итераторы работают эффективно и используя знание о внутренней структуре объектов, а не вызывают публичные методы объекта табулированной функции.

Изменения в `TabulatedFunction`:

```
package functions;

import java.util.Iterator;

public interface TabulatedFunction extends Function, Iterable<FunctionPoint>, Cloneable {
    // Методы для работы с точками табулированной функции
    int getPointsCount();
    FunctionPoint getPoint(int index);
    void setPoint(int index, FunctionPoint point) throws InappropriateFunctionPointException;
    double getPointX(int index);
    void setPointX(int index, double x) throws InappropriateFunctionPointException;
    double getPointY(int index);
    void setPointY(int index, double y);
    void deletePoint(int index);
    void addPoint(FunctionPoint point) throws InappropriateFunctionPointException;
    @Override
    Iterator<FunctionPoint> iterator();
    // Метод clone() добавлен в интерфейс
    Object clone();
}
```

Тестирование в классе `Main`:

```
// 1. Создаем тестовые точки
System.out.println("\n1. Создание тестовых точек...");
FunctionPoint[] points = {
    new FunctionPoint(x: 0.0, y: 0.0),
    new FunctionPoint(x: 1.0, y: 1.0),
    new FunctionPoint(x: 2.0, y: 4.0),
    new FunctionPoint(x: 3.0, y: 9.0),
    new FunctionPoint(x: 4.0, y: 16.0),
    new FunctionPoint(x: 5.0, y: 25.0)
};

System.out.println("    Создано " + points.length + " тестовых точек:");
for (int i = 0; i < points.length; i++) {
    System.out.println("    " + i + ": " + points[i]);
}
```

```
// 2. Тестирование ArrayTabulatedFunction
System.out.println("\n2. Тестирование ArrayTabulatedFunction:");
System.out.println("    Создание ArrayTabulatedFunction...");
TabulatedFunction arrayFunction = new ArrayTabulatedFunction(points);

System.out.println("    Использование цикла for-each:");
int count = 0;
for (FunctionPoint point : arrayFunction) {
    System.out.println("        Точка " + count + ": " + point);
    count++;
}

System.out.println("    Проверка сохранения инкапсуляции:");
// Получаем первую точку через итератор
FunctionPoint firstPoint = null;
for (FunctionPoint point : arrayFunction) {
    firstPoint = point;
    break;
}

if (firstPoint != null) {
    System.out.println("        Получена точка: " + firstPoint);

    // Попробуем изменить полученную точку
    firstPoint.setX(100.0);
    firstPoint.setY(200.0);
    System.out.println("        Измененная копия точки: " + firstPoint);

    // Проверяем, что исходная функция не изменилась
    System.out.println("        Первая точка в исходной функции: " + arrayFunction.getPoint(index: 0));
    System.out.println("        !Инкапсуляция не нарушена (точка скопирована)");
}
}
```

```
// 3. Тестирование LinkedListTabulatedFunction
System.out.println("\n3. Тестирование LinkedListTabulatedFunction:");
System.out.println("    Создание LinkedListTabulatedFunction...");
TabulatedFunction linkedListFunction = new LinkedListTabulatedFunction(points);

System.out.println("    Использование цикла for-each:");
count = 0;
for (FunctionPoint point : linkedListFunction) {
    System.out.println("        Точка " + count + ": " + point);
    count++;
}
}
```

```

System.out.println("\n4. Тестирование исключений итератора:");

System.out.println("    а) Тестирование NoSuchElementException:");
try {
    // Создаем небольшую функцию и получаем из нее больше точек, чем есть
    FunctionPoint[] fewPoints = {
        new FunctionPoint(x: 0, y: 0),
        new FunctionPoint(x: 1, y: 1)
    };
    TabulatedFunction smallFunction = new ArrayTabulatedFunction(fewPoints);

    int iteration = 0;
    for (FunctionPoint point : smallFunction) {
        System.out.println("        Итерация " + iteration + ": " + point);
        iteration++;
    }
    System.out.println("        !NoSuchElementException не возникло (все в порядке)");
} catch (Exception e) {
    System.out.println("        !Исключение: " + e.getClass().getSimpleName() + " - " + e.getMessage());
}

System.out.println("\n    б) Тестирование UnsupportedOperationException при вызове remove():");
try {
    TabulatedFunction testFunction = new ArrayTabulatedFunction(points);
    java.util.Iterator<FunctionPoint> iterator = testFunction.iterator();

    if (iterator.hasNext()) {
        iterator.next(); // Получаем элемент
        iterator.remove(); // Пробуем удалить
    }
} catch (UnsupportedOperationException e) {
    System.out.println("        !Корректно выброшено UnsupportedOperationException: " + e.getMessage());
} catch (Exception e) {
    System.out.println("        !Неожиданное исключение: " + e.getClass().getSimpleName() + " - " + e.getl
}

```

```

// 5. Сравнение работы итераторов двух реализаций
System.out.println("\n5. Сравнение работы ArrayTabulatedFunction и LinkedListTabulatedFunction:");

System.out.println("    а) ArrayTabulatedFunction (через for-each):");
for (FunctionPoint point : arrayFunction) {
    System.out.println("        " + point);
}

System.out.println("\n    б) ArrayTabulatedFunction (через явный итератор:");
java.util.Iterator<FunctionPoint> arrayIterator = arrayFunction.iterator();
while (arrayIterator.hasNext()) {
    System.out.println("        " + arrayIterator.next());
}

System.out.println("\n    в) LinkedListTabulatedFunction (через for-each):");
for (FunctionPoint point : linkedListFunction) {
    System.out.println("        " + point);
}

System.out.println("\n    г) LinkedListTabulatedFunction (через явный итератор:");
java.util.Iterator<FunctionPoint> linkedListIterator = linkedListFunction.iterator();
while (linkedListIterator.hasNext()) {
    System.out.println("        " + linkedListIterator.next());
}

```

```
// 6. Тестирование с различными типами функций
System.out.println("\n6. Тестирование с различными типами функций");

System.out.println("    а) Функция с одной точкой (должна быть как минимум 2):");
try {
    FunctionPoint[] onePoint = { new FunctionPoint(x: 0, y: 0) };
    TabulatedFunction singlePointFunc = new ArrayTabulatedFunction(onePoint);
} catch (IllegalArgumentException e) {
    System.out.println("        !Корректно выброшено исключение: " + e.getMessage());
}

System.out.println("\n    б) Функция с большим количеством точек:");
FunctionPoint[] manyPoints = new FunctionPoint[20];
for (int i = 0; i < 20; i++) {
    manyPoints[i] = new FunctionPoint(i, y: i * i);
}

TabulatedFunction largeArrayFunc = new ArrayTabulatedFunction(manyPoints);
System.out.println("        Создана ArrayTabulatedFunction с 20 точками");
System.out.println("        Обход через for-each (первые 5 точек:");
int printed = 0;
for (FunctionPoint point : largeArrayFunc) {
    if (printed < 5) {
        System.out.println("            " + point);
        printed++;
    } else {
        break;
    }
}
System.out.println("        ... и еще " + (20 - printed) + " точек");
```

```
// 7. Проверка, что итераторы работают независимо
System.out.println("\n7. Проверка независимости итераторов");

TabulatedFunction testFunc = new ArrayTabulatedFunction(points);

System.out.println("    Создаем два независимых итератора");
java.util.Iterator<FunctionPoint> iterator1 = testFunc.iterator();
java.util.Iterator<FunctionPoint> iterator2 = testFunc.iterator();

System.out.println("    Итератор 1, первые 3 точки:");
for (int i = 0; i < 3 && iterator1.hasNext(); i++) {
    System.out.println("        " + iterator1.next());
}

System.out.println("    Итератор 2, все точки:");
while (iterator2.hasNext()) {
    System.out.println("        " + iterator2.next());
}

System.out.println("    Итератор 1, оставшиеся точки:");
while (iterator1.hasNext()) {
    System.out.println("        " + iterator1.next());
}
```

```
// 8. Тестирование вложенных циклов
System.out.println("\n8. Тестирование вложенных циклов:");
System.out.println("    Вложенный обход одной и той же функции:");

int outerCount = 0;
for (FunctionPoint outerPoint : testFunc) {
    System.out.println("    Внешний цикл, точка " + outerCount + ": " + outerPoint);

    int innerCount = 0;
    for (FunctionPoint innerPoint : testFunc) {
        System.out.println("        Внутренний цикл, точка " + innerCount + ": " + innerPoint);
        innerCount++;
        if (innerCount >= 2) break; // Ограничиваем внутренний цикл для читаемости
    }

    outerCount++;
    if (outerCount >= 2) break; // Ограничиваем внешний цикл
}
}
```

```
// 9. Итоговая проверка
System.out.println("\n9. Итоговая проверка:");
System.out.println("    Проверяем, что обе реализации дадут одинаковый результат при обходе:");

boolean allPointsEqual = true;
java.util.Iterator<FunctionPoint> arrayIter = arrayFunction.iterator();
java.util.Iterator<FunctionPoint> linkedListIter = linkedListFunction.iterator();

while (arrayIter.hasNext() && linkedListIter.hasNext()) {
    FunctionPoint arrayPoint = arrayIter.next();
    FunctionPoint linkedListPoint = linkedListIter.next();

    if (!arrayPoint.equals(linkedListPoint)) {
        allPointsEqual = false;
        System.out.println("        !Точки не совпадают: " + arrayPoint + " != " + linkedListPoint);
    }
}

if (allPointsEqual && !arrayIter.hasNext() && !linkedListIter.hasNext()) {
    System.out.println("        !Все точки совпадают, итераторы работают корректно");
} else {
    System.out.println("        !Количество точек или их значения не совпадают");
}

System.out.println("\n=== Тестирование завершено ===");
```

Полученный вывод программы, говорит о её правильно работе:

=== Лабораторная работа №7. Задание 1: Тестирование итераторов ===

1. Создание тестовых точек...

Создано 6 тестовых точек:

0: (0.0; 0.0)
1: (1.0; 1.0)
2: (2.0; 4.0)
3: (3.0; 9.0)
4: (4.0; 16.0)
5: (5.0; 25.0)

2. Тестирование ArrayTabulatedFunction:

Создание ArrayTabulatedFunction...

Использование цикла for-each:

Точка 0: (0.0; 0.0)
Точка 1: (1.0; 1.0)
Точка 2: (2.0; 4.0)
Точка 3: (3.0; 9.0)
Точка 4: (4.0; 16.0)
Точка 5: (5.0; 25.0)

Проверка сохранения инкапсуляции:

Получена точка: (0.0; 0.0)
Измененная копия точки: (100.0; 200.0)
Первая точка в исходной функции: (0.0; 0.0)
!Инкапсуляция не нарушена (точка скопирована)

3. Тестирование LinkedListTabulatedFunction:

Создание LinkedListTabulatedFunction...

Использование цикла for-each:

Точка 0: (0.0; 0.0)
Точка 1: (1.0; 1.0)
Точка 2: (2.0; 4.0)
Точка 3: (3.0; 9.0)
Точка 4: (4.0; 16.0)
Точка 5: (5.0; 25.0)

4. Тестирование исключений итератора:

а) Тестирование NoSuchElementException:

Итерация 0: (0.0; 0.0)
Итерация 1: (1.0; 1.0)
!NoSuchElementException не возникло (все в порядке)

б) Тестирование UnsupportedOperationException при вызове remove():

!Корректно выброшено UnsupportedOperationException: Remove operation is not supported

5. Сравнение работы ArrayTabulatedFunction и LinkedListTabulatedFunction:

а) ArrayTabulatedFunction (через for-each):

(0.0; 0.0)
(1.0; 1.0)
(2.0; 4.0)
(3.0; 9.0)
(4.0; 16.0)
(5.0; 25.0)

б) ArrayTabulatedFunction (через явный итератор):

(0.0; 0.0)
(1.0; 1.0)
(2.0; 4.0)
(3.0; 9.0)
(4.0; 16.0)
(5.0; 25.0)

в) LinkedListTabulatedFunction (через for-each):

(0.0; 0.0)
(1.0; 1.0)
(2.0; 4.0)
(3.0; 9.0)
(4.0; 16.0)
(5.0; 25.0)

г) LinkedListTabulatedFunction (через явный итератор):

(0.0; 0.0)
(1.0; 1.0)
(2.0; 4.0)
(3.0; 9.0)
(4.0; 16.0)
(5.0; 25.0)

```
6. Тестирование с различными типами функций:
а) Функция с одной точкой (должна быть как минимум 2):
    !Корректно выброшено исключение: Необходимо минимум 2 точки

б) Функция с большим количеством точек:
    Создана ArrayTabulatedFunction с 20 точками
    Обход через for-each (первые 5 точек):
        (0.0; 0.0)
        (1.0; 1.0)
        (2.0; 4.0)
        (3.0; 9.0)
        (4.0; 16.0)
    ... и еще 15 точек

7. Проверка независимости итераторов:
    Создаем два независимых итератора:
    Итератор 1, первые 3 точки:
        (0.0; 0.0)
        (1.0; 1.0)
        (2.0; 4.0)
    Итератор 2, все точки:
        (0.0; 0.0)
        (1.0; 1.0)
        (2.0; 4.0)
        (3.0; 9.0)
        (4.0; 16.0)
        (5.0; 25.0)
    Итератор 1, оставшиеся точки:
        (3.0; 9.0)
        (4.0; 16.0)
        (5.0; 25.0)
```

```
8. Тестирование вложенных циклов:
    Вложенный обход одной и той же функции:
    Внешний цикл, точка 0: (0.0; 0.0)
        Внутренний цикл, точка 0: (0.0; 0.0)
        Внутренний цикл, точка 1: (1.0; 1.0)
    Внешний цикл, точка 1: (1.0; 1.0)
        Внутренний цикл, точка 0: (0.0; 0.0)
        Внутренний цикл, точка 1: (1.0; 1.0)

9. Итоговая проверка:
    Проверяем, что обе реализации дают одинаковый результат при обходе:
    !Все точки совпадают, итераторы работают корректно

=== Тестирование завершено ===
```

Задание 2

Цель: создать механизм фабрик для создания объектов табулированных функций с возможностью динамической замены фабрики.

В пакете `functions` описан базовый интерфейс фабрик табулированных функций `TabulatedFunctionFactory`. Интерфейс объявляет три перегруженных метода `TabulatedFunction createTabulatedFunction()`, параметры которых соответствуют параметрам конструкторов классов табулированных функций.

В классе `TabulatedFunctions` объявлено приватное статическое поле типа `TabulatedFunctionFactory` и проинициализировано объектом одного из описанных классов фабрик. Также объявлен метод `setTabulatedFunctionFactory()`, позволяющий заменить объект фабрики.

Ещё в классе `TabulatedFunctions` описано три перегруженных метода `TabulatedFunction createTabulatedFunction()`, возвращающих объекты табулированных функций, созданных с помощью текущей фабрики. Параметры методов соответствуют параметрам методов фабрики.

Создание интерфейса `TabulatedFunctionFactory`:

```
package functions;

public interface TabulatedFunctionFactory {
    TabulatedFunction createTabulatedFunction(double leftX, double rightX, int pointsCount);
    TabulatedFunction createTabulatedFunction(double leftX, double rightX, double[] values);
    TabulatedFunction createTabulatedFunction(FunctionPoint[] points);
}
```

Изменения в `ArrayTabulatedFunction`:

```
public static class ArrayTabulatedFunctionFactory implements TabulatedFunctionFactory {
    @Override
    public TabulatedFunction createTabulatedFunction(double leftX, double rightX, int pointsCount) {
        return new ArrayTabulatedFunction(leftX, rightX, pointsCount);
    }

    @Override
    public TabulatedFunction createTabulatedFunction(double leftX, double rightX, double[] values) {
        return new ArrayTabulatedFunction(leftX, rightX, values);
    }

    @Override
    public TabulatedFunction createTabulatedFunction(FunctionPoint[] points) {
        return new ArrayTabulatedFunction(points);
    }
}
```

Изменения в `LinkedListTabulatedFunction`:

```
public static class LinkedListTabulatedFunctionFactory implements TabulatedFunctionFactory { no usages
    @Override no usages
    public TabulatedFunction createTabulatedFunction(double leftX, double rightX, int pointsCount) {
        return new LinkedListTabulatedFunction(leftX, rightX, pointsCount);
    }

    @Override no usages
    public TabulatedFunction createTabulatedFunction(double leftX, double rightX, double[] values) {
        return new LinkedListTabulatedFunction(leftX, rightX, values);
    }

    @Override no usages
    public TabulatedFunction createTabulatedFunction(FunctionPoint[] points) {
        return new LinkedListTabulatedFunction(points);
    }
}
```

Изменения в TabulatedFunctions:

```

public class TabulatedFunctions { 1 usage

    // Приватный конструктор, чтобы нельзя было создать экземпляр класса
    private TabulatedFunctions() { no usages
        throw new UnsupportedOperationException("Нельзя создать экземпляр класса TabulatedFunctions");
    }

    // Статическое поле для хранения фабрики табулированных функций
    private static TabulatedFunctionFactory factory = 6 usages
        new ArrayTabulatedFunction.ArrayTabulatedFunctionFactory();

    // Метод для установки фабрики
    public static void setTabulatedFunctionFactory(TabulatedFunctionFactory factory) { no usages
        if (factory == null) {
            throw new IllegalArgumentException("Фабрика не может быть null");
        }
        TabulatedFunctions.factory = factory;
    }

    // ===== Методы создания через фабрику =====

    /**
     * Создает табулированную функцию с равномерным распределением точек.
     * @param leftX левая граница области определения
     * @param rightX правая граница области определения
     * @param pointsCount количество точек
     * @return созданная табулированная функция
     */
    public static TabulatedFunction createTabulatedFunction(double leftX, double rightX, int pointsCount) {
        if (leftX >= rightX) {
            throw new IllegalArgumentException("Левая граница должна быть меньше правой");
        }
        if (pointsCount < 2) {
            throw new IllegalArgumentException("Количество точек должно быть не менее 2");
        }
        return factory.createTabulatedFunction(leftX, rightX, pointsCount);
    }
}

```

Остальные методы в данном классе также должны использовать фабрику.

Тестирование в методе main класса Main:

```

System.out.println("\n== Задание 2: Тестирование фабрик ==");

Function f = new functions.basic.Cos();
TabulatedFunction tf;

// Использование фабрики по умолчанию (ArrayTabulatedFunction)
tf = TabulatedFunctions.tabulate(f, leftX: 0, Math.PI, pointsCount: 11);
System.out.println("Фабрика по умолчанию: " + tf.getClass().getSimpleName());

// Установка фабрики для LinkedListTabulatedFunction
TabulatedFunctions.setTabulatedFunctionFactory(
    new LinkedListTabulatedFunction.LinkedListTabulatedFunctionFactory()
);
tf = TabulatedFunctions.tabulate(f, leftX: 0, Math.PI, pointsCount: 11);
System.out.println("LinkedList фабрика: " + tf.getClass().getSimpleName());

// Возврат к фабрике ArrayTabulatedFunction
TabulatedFunctions.setTabulatedFunctionFactory(
    new ArrayTabulatedFunction.ArrayTabulatedFunctionFactory()
);
tf = TabulatedFunctions.tabulate(f, leftX: 0, Math.PI, pointsCount: 11);
System.out.println("Array фабрика снова: " + tf.getClass().getSimpleName());

```

Проводим проверку фабрик, как написано в задании:

```

1. Тестирование фабрики по умолчанию (ArrayTabulatedFunction):
  Тип созданной функции: ArrayTabulatedFunction
  Значения функции:
    (0.0; 0.0)
    (0.7853981633974483; 0.7071067811865475)
    (1.5707963267948966; 1.0)
    (2.356194490192345; 0.7071067811865476)
    (3.141592653589793; 1.2246467991473532E-16)
  Создание через createTabulatedFunction:
  Тип: ArrayTabulatedFunction
  Значения: {(0.0; 0.0), (5.0; 0.0), (10.0; 0.0)}

2. Смена фабрики на LinkedListTabulatedFunction:
  Тип созданной функции: LinkedListTabulatedFunction
  Значения функции:
    (0.0; 1.0)
    (0.7853981633974483; 0.7071067811865476)
    (1.5707963267948966; 6.123233995736766E-17)
    (2.356194490192345; -0.7071067811865475)
    (3.141592653589793; -1.0)
  Текущая фабрика: LinkedListTabulatedFunctionFactory

3. Возврат к фабрике ArrayTabulatedFunction:
  Тип созданной функции: ArrayTabulatedFunction
  Значения функции (тангенс):
    (0.0; 0.0)
    (0.3676990816987241; 0.38521845377303177)
    (0.7353981633974482; 0.9046862463150047)
    (1.1030972450961722; 1.9799053523951764)
    (1.4707963267948965; 9.966644423259222)

4. Тестирование методов TabulatedFunctions с фабрикой:
  а) Создание функции с равномерным распределением:
    Тип: ArrayTabulatedFunction
    Значения: {(0.0; 0.0), (2.5; 0.0), (5.0; 0.0), (7.5; 0.0), (10.0; 0.0)}
  б) Создание функции с заданными значениями:
    Тип: ArrayTabulatedFunction
    Значения: {(0.0; 0.0), (1.0; 1.0), (2.0; 4.0), (3.0; 9.0), (4.0; 16.0)}
  в) Создание функции из массива точек:
    Тип: ArrayTabulatedFunction
    Значения: {(0.0; 0.0), (2.0; 8.0), (4.0; 64.0)}
  г) Тестирование ввода/вывода с фабрикой:
    Бинарный ввод/вывод:   ОК
    Текстовый ввод/вывод:  ОК

```

Программа работает нормально.

Задание 3

Цель: добавить возможность создания объектов табулированных функций с использованием рефлексии, передавая класс в качестве параметра.

В классе `TabulatedFunctions` добавлены ещё три перегруженных версии метода `createTabulatedFunction()`. Их параметры повторяют параметры трёх аналогичных методов, основанных на использовании фабрики, но также эти методы получают ссылку типа `Class` на описание класса, объект которого требуется создать. Методы написаны так, чтобы в эти методы можно было передать только ссылки на классы, реализующие интерфейс `TabulatedFunction`.

```
// ===== МЕТОДЫ СОЗДАНИЯ ЧЕРЕЗ РЕФЛЕКСИЮ (ЗАДАНИЕ 3) =====

public static TabulatedFunction createTabulatedFunction( 1 usage
    Class<? extends TabulatedFunction> functionClass,
    double leftX, double rightX, int pointsCount) {

    validateClassParameter(functionClass);
    validateCreateParameters(leftX, rightX, pointsCount);

    try {
        Constructor<? extends TabulatedFunction> constructor =
            functionClass.getConstructor(double.class, double.class, int.class);
        return constructor.newInstance(leftX, rightX, pointsCount);
    } catch (NoSuchMethodException e) {
        throw new IllegalArgumentException("Конструктор (double, double, int) не найден в классе " +
            functionClass.getSimpleName(), e);
    } catch (InstantiationException e) {
        throw new IllegalArgumentException("Не удалось создать экземпляр класса " +
            functionClass.getSimpleName(), e);
    } catch (IllegalAccessException e) {
        throw new IllegalArgumentException("Нет доступа к конструктору класса " +
            functionClass.getSimpleName(), e);
    } catch (InvocationTargetException e) {
        Throwable cause = e.getCause();
        throw new IllegalArgumentException("Исключение в конструкторе класса " +
            functionClass.getSimpleName() + ": " +
            (cause != null ? cause.getMessage() : e.getMessage()), cause);
    }
}
```

```
public static TabulatedFunction createTabulatedFunction( 2 usages
    Class<? extends TabulatedFunction> functionClass,
    double leftX, double rightX, double[] values) {

    validateClassParameter(functionClass);
    validateCreateParameters(leftX, rightX, values);

    try {
        Constructor<? extends TabulatedFunction> constructor =
            functionClass.getConstructor(double.class, double.class, double[].class);
        return constructor.newInstance(leftX, rightX, values);
    } catch (NoSuchMethodException e) {
        throw new IllegalArgumentException("Конструктор (double, double, double[]) не найден в классе " +
            functionClass.getSimpleName(), e);
    } catch (Exception e) {
        handleReflectionException(e, functionClass);
        return null; // Эта строка никогда не выполнится
    }
}
```

```

public static TabulatedFunction createTabulatedFunction( 1 usage
    Class<? extends TabulatedFunction> functionClass,
    FunctionPoint[] points) {

    validateClassParameter(functionClass);
    validateCreateParameters(points);

    try {
        Constructor<? extends TabulatedFunction> constructor =
            functionClass.getConstructor(FunctionPoint[].class);
        return constructor.newInstance((Object) points);
    } catch (NoSuchMethodException e) {
        throw new IllegalArgumentException("Конструктор (FunctionPoint[]) не найден в классе " +
            functionClass.getSimpleName(), e);
    } catch (Exception e) {
        handleReflectionException(e, functionClass);
        return null;
    }
}

```

В класс Main были внесены изменения для проверки корректности работы написанной программы:

```

private static void testReflection() { 1 usage
    System.out.println("\n=== ЗАДАНИЕ 3: Тестирование рефлексивного создания объектов ===\n");

    // 1. Создание ArrayTabulatedFunction через рефлексию
    System.out.println("1. Создание ArrayTabulatedFunction через рефлексию:");
    testReflectionArrayTabulatedFunction();

    // 2. Создание LinkedListTabulatedFunction через рефлексию
    System.out.println("\n2. Создание LinkedListTabulatedFunction через рефлексию:");
    testReflectionLinkedListTabulatedFunction();

    // 3. Тестирование метода tabulate с рефлексией
    System.out.println("\n3. Табулирование с использованием рефлексии:");
    testReflectionTabulate();

    System.out.println("\n Задание 3 завершено успешно!");
}

```

В результате запуска программы получается следующий вывод, который говорит о том, что программа работает корректно:

Задание 2 завершено успешно!

=== ЗАДАНИЕ 3: Тестирование рефлексивного создания объектов ===

1. Создание ArrayTabulatedFunction через рефлексия:

- а) Создание с базовыми параметрами:
Тип: ArrayTabulatedFunction
Значения: {(0.0; 0.0), (5.0; 0.0), (10.0; 0.0)}
- б) Создание с массивом значений:
Тип: ArrayTabulatedFunction
Значения: {(0.0; 0.0), (5.0; 10.0), (10.0; 20.0)}
- в) Создание из массива точек:
Тип: ArrayTabulatedFunction
Значения: {(0.0; 0.0), (5.0; 25.0), (10.0; 100.0)}

2. Создание LinkedListTabulatedFunction через рефлексия:

- а) Создание с базовыми параметрами:
Тип: LinkedListTabulatedFunction
Значения: {(0.0; 0.0), (5.0; 0.0), (10.0; 0.0)}
- б) Создание с массивом значений:
Тип: LinkedListTabulatedFunction
Значения: {(0.0; 0.0), (5.0; 10.0), (10.0; 20.0)}
- в) Создание из массива точек:
Тип: LinkedListTabulatedFunction
Значения: {(0.0; 0.0), (5.0; 125.0), (10.0; 1000.0)}

3. Табулирование с использованием рефлексии:

- а) Табулирование синуса в ArrayTabulatedFunction:
Тип: ArrayTabulatedFunction
Значения: {(0.0; 0.0), (0.7853981633974483; 0.7071067811865475), (1.5707963267948966; 1.0), (2.356194490192345; 0.7071067811865476), 6)}
- б) Табулирование экспоненты в LinkedListTabulatedFunction:
Тип: LinkedListTabulatedFunction
Значения: {(0.0; 1.0), (0.6666666666666666; 1.9477340410546757), (1.3333333333333333; 3.7936678946831774), (2.0; 7.38905609893065)}
- в) Сравнение с обычным табулированием:
Обычное табулирование: ArrayTabulatedFunction
Рефлексивное табулирование: ArrayTabulatedFunction
Функции равны: Да

Задание 3 завершено успешно!